Service Booking & Consulting Platform

Phase 2 — Complete Project Reference Guide

Course	EECS 3311 — Software Design
University	York University — Lassonde School of Engineering
Stack	Java 17 | Maven | PostgreSQL | HTML/CSS/JS | Docker
GitHub	https://github.com/AliSheikhRabiei/EECS3311-Service-Booking-Platform
Ali Sheikh Rabiei	218475095
Yasamin Kheirkhahan	220067328

1. Phase 2 Overview

Phase 2 extends the Phase 1 terminal-based Java backend into a complete, deployable web application. The core Phase 1 business logic is kept entirely intact. Phase 2 adds five major layers on top of it:

What was added	Description
PostgreSQL database	Replaces all in-memory HashMaps. All data survives server restarts.
HTTP REST API	JSON endpoints using Java's built-in HttpServer. Token-based login for all three roles.
Web frontend	Multi-page HTML/CSS/JS app. Dashboards for Client, Consultant, and Admin.
AI Customer Assistant	Chatbot in the Client dashboard. Backend calls an external LLM API securely.
Docker deployment	Three containers started with a single docker-compose up command.

Core design principle:
Phase 1 logic was NOT rewritten. Every HTTP handler calls the same BookingService, PaymentService, AvailabilityService, and ServiceCatalog from Phase 1. The new database layer sits underneath the existing service layer, not instead of it.

2. Complete File Change Summary

2.1 Files Unchanged from Phase 1

Every file in the packages below is identical to Phase 1. They were copied without any modification.

Package	Files (all unchanged)
domain/	User.java, Client.java, Consultant.java, Admin.java, Service.java, TimeSlot.java, Booking.java, RegistrationStatus.java, Decision.java
state/	BookingState.java, AbstractBookingState.java, RequestedState.java, ConfirmedState.java, PendingPaymentState.java, PaidState.java, RejectedState.java, CancelledState.java, CompletedState.java
payment/ (logic)	PaymentHandler.java, AbstractPaymentHandler.java, ValidateMethodHandler.java, ProcessPaymentHandler.java, MarkPaidHandler.java, PaymentService.java, PaymentMethodService.java, PaymentTransaction.java, PaymentStatus.java, Receipt.java, PaymentMethod.java, PayPalMethod.java
policy/ (originals)	PolicyManager.java, CancellationPolicy.java, DefaultCancellationPolicy.java, RefundPolicy.java, DefaultRefundPolicy.java, FullRefundPolicy.java, NotificationPolicy.java, DefaultNotificationPolicy.java, PricingStrategy.java, DefaultPricingStrategy.java
repository/	BookingRepository.java, ConsultantRepository.java (kept as base classes — extended by DB layer)
service/	ServiceCatalog.java, AvailabilityService.java, RegistrationService.java
notify/	NotificationService.java
app/	Main.java — terminal demo still works via mvn exec:java
test/	All 13 JUnit 5 test files — unchanged

2.2 Files Modified from Phase 1

pom.xml

Two new runtime dependencies were added. The build plugin changed from maven-jar-plugin to maven-shade-plugin to produce a fat JAR (includes all dependencies so Docker can run it with java -jar). The default mainClass changed to HttpServerLauncher.

What changed	Before (Phase 1)	After (Phase 2)
Gson dependency	Not present	com.google.code.gson : gson : 2.10.1
PostgreSQL driver	Not present	org.postgresql : postgresql : 42.7.3
Build plugin	maven-jar-plugin	maven-shade-plugin 3.5.1 (fat JAR)
Default mainClass	com.platform.app.Main	com.platform.http.HttpServerLauncher

Why the fat JAR matters: the Docker runtime image only needs a JRE, not full Maven. Gson and the PostgreSQL driver are bundled inside the JAR.

payment/CreditCardMethod.java — 3 getter methods added

Zero logic changed. Three public getter methods were added so the database layer can read the card fields when persisting to the payment_methods table.

```
public String getCardNumber() { return cardNumber; }
public String getExpiry()     { return expiry; }
public String getCvv()        { return cvv; }
```

payment/DebitCardMethod.java — 2 getter methods added

```
public String getCardNumber() { return cardNumber; }
public String getExpiry()     { return expiry; }
```

payment/BankTransferMethod.java — 2 getter methods added

```
public String getAccountNumber() { return accountNumber; }
public String getRoutingNumber() { return routingNumber; }
```

Why these getters are needed: the fields are private final in Phase 1. The PaymentMethodRepository serializes them as JSON to store in the database. Without getters there is no way to read them without changing the original design.

service/BookingService.java — Booking ID generation changed

This is the only logic change in any Phase 1 service class.

	Phase 1	Phase 2
ID type	AtomicInteger counter starting at 1	UUID (random)
Example ID	BK-1	BK-A3F2E891
Why changed	Counter resets to 1 on restart. With a database, restarting and creating a booking generates BK-1 again — but that ID already exists, causing a primary key conflict crash.	UUIDs are globally unique across every restart.

2.3 New Files in Existing Packages

policy/NoRefundPolicy.java — NEW

A named class implementing RefundPolicy that returns 0.0 (no refund). Replaces the anonymous lambda (tx, now) -> 0.0 that was used previously.

Why it was needed: Java lambda instances have no meaningful class name. getClass().getSimpleName() on a lambda returns an unreadable JVM-internal string like AdminHandler$$Lambda$98/0x000070aaaa0be470. The admin policies page was showing this instead of a clean name. Using a named class fixes the display permanently.

policy/StrictCancellationPolicy.java — NEW

A named class implementing CancellationPolicy. canCancel() always returns false and cancellationFee() returns the full service price. Replaces the anonymous class used previously. Same reason as NoRefundPolicy — fixes the admin policy display.

3. New Package: com.platform.db/

This package contains the JDBC persistence layer. It sits between the HTTP handlers and the Phase 1 service layer. The Phase 1 services still use their in-memory state. On every write operation the DB layer also persists to PostgreSQL. On server startup the DB layer reloads all existing data back into the in-memory services so the Phase 1 logic works identically.

Database.java

Connection factory and schema initialiser. Single source of truth for database connection details.

Method	What it does
getConnection()	Returns a fresh JDBC connection from DriverManager. Every repository method calls this and closes it in a try-with-resources block to avoid connection leaks.
initialize()	Reads schema.sql from the classpath, strips comment lines per statement chunk, then executes each CREATE TABLE IF NOT EXISTS statement. Called once on server startup. Safe to re-run on every restart.

Environment variables read (Docker injects these):

- DB_URL — defaults to jdbc:postgresql://localhost:5432/booking_platform
- DB_USER — defaults to postgres
- DB_PASSWORD — defaults to postgres

Important SQL parsing fix:
The initialize() method strips comment-only lines from each SQL chunk before checking if the chunk is empty. Without this fix, the first chunk (which starts with -- header comments followed by CREATE TABLE users) was silently skipped, causing 'relation users does not exist' errors on every startup.

UserRepository.java

All CRUD operations for the users and consultants tables. Reconstructs Client and Consultant domain objects from database rows.

Method	What it does

saveUser(id, name, email, hash, role)	Inserts or updates a user row. Uses ON CONFLICT (id) DO UPDATE so it is safe to call repeatedly.
saveConsultantProfile(userId, bio, status)	Inserts or updates the consultants table row linked to a user.
updateConsultantStatus(userId, status)	Updates only the registration_status column. Called by AdminHandler after approve/reject.
findByEmail(email)	Returns a UserRow data holder. Used by AuthService.login() to verify credentials.
loadClient(id)	Constructs and returns a Client domain object from the DB. Returns null if not found or role is not CLIENT.
loadConsultant(id)	Constructs and returns a Consultant domain object. Sets registrationStatus from the consultants table.
findAllConsultants()	Returns all consultants as domain objects. Used by AdminHandler.
emailExists(email)	Boolean check. Used by AuthService to enforce unique email addresses on registration.

ServiceRepository.java

CRUD for the services table. Every findById and findAll call also loads the owning Consultant via UserRepository.loadConsultant() to reconstruct the full Service object graph that the Phase 1 ServiceCatalog expects.

Method	What it does
save(Service s)	Inserts or updates a service row.
findById(id)	Loads one service and its consultant.
findAll()	Loads all services ordered by title. Used by AppContext on startup to repopulate ServiceCatalog.
findByConsultantId(id)	Loads all services for one consultant.

SlotRepository.java

CRUD for the time_slots table. Each slot has a UUID primary key in the DB. The Phase 1 TimeSlot domain object uses start.toString() as its slotId.

Method	What it does
save(consultantId, slot)	Generates a UUID, inserts the slot row, returns the UUID.
updateAvailability(uuid, available)	Updates the is_available column. Called when a booking is created (false) or cancelled/rejected (true).
delete(uuid)	Removes a slot. Only allowed if is_available = true.

findByUuid(uuid)	Returns a SlotRow by UUID. Used by BookingsHandler to resolve the slot when creating a booking.
findByConsultantAndStart(id, start)	Looks up a slot by consultant and start time. Used to find the DB UUID when saving a booking.
findByConsultantId(id)	Returns all slots for a consultant ordered by start time.
findAll()	Returns every slot across all consultants. Used by AppContext on startup to repopulate AvailabilityService.

DbBookingRepository.java

Database-backed replacement for the Phase 1 BookingRepository. Extends BookingRepository so it can be passed anywhere the parent type is expected. The parent's in-memory HashMap is never used — all operations go to PostgreSQL.

Method	What it does
save(Booking b)	Inserts a new booking or updates its status. Looks up the slot's DB UUID via SlotRepository to populate the slot_id foreign key.
updateStatus(bookingId, status)	Updates ONLY the status column. Called by HTTP handlers after every state transition (confirm, reject, complete, cancel, pay). More efficient than a full save().
findById(id)	Reconstructs a full Booking object from the DB row by loading client, service, and slot from their repositories, then restoring the correct BookingState via BookingStateFactory.
findByClient(c)	Returns all bookings for a specific client.
findByConsultant(con)	Returns all bookings for services offered by a specific consultant.
findAll()	Returns all bookings ordered by created_at.

DbConsultantRepository.java

Database-backed replacement for the Phase 1 ConsultantRepository. Extends it so the existing Admin domain class (which holds a ConsultantRepository reference) continues to work. All three methods (save, findById, findAll) delegate to UserRepository.

PaymentMethodRepository.java

Persists and loads payment methods from the payment_methods table. Type-specific fields (card number, expiry, CVV, etc.) are serialized as a JSON string in the details column.

Method	What it does

save(clientId, method)	Serializes the method's type-specific fields to JSON, then inserts or updates the row.
delete(clientId, methodId)	Removes a method by id, scoped to the client so one client cannot delete another's methods.
findByClientId(clientId, owner)	Loads all methods for a client, deserializing the details JSON back into the correct PaymentMethod subclass.

Why JSON in the details column: each payment method subclass has different fields. Storing type-specific data as JSON in one column is cleaner than nullable columns for every possible field across all types.

PaymentTransactionRepository.java

Persists PaymentTransaction objects to the payment_transactions table and retrieves them for payment history.

Method	What it does
save(tx)	Inserts or updates a transaction row.
findByClientId(clientId)	Returns all transactions for a client joined through the bookings table. Returns TransactionRow objects (not full domain objects) for efficiency.
findByBookingId(id)	Returns all transactions for one booking. Used to find the original SUCCESS transaction when processing a refund.

BookingStateFactory.java

Converts between the string stored in the DB (e.g. 'CONFIRMED') and the concrete BookingState subclass instances used by the domain Booking object.

Why this is needed: when a booking is loaded from the database, DbBookingRepository needs to call booking.setState(correctStateInstance). The Booking constructor always sets RequestedState — without this factory, every booking loaded from DB would incorrectly appear as REQUESTED regardless of its actual state.

Method	What it does
fromString(status)	Returns the correct state instance for a given status string. Handles both PENDING_PAYMENT and PENDINGPAYMENT for robustness.
toDbString(state)	Returns the canonical DB string for a given state object. Used whenever a handler needs to update the status column.

4. New Package: com.platform.auth/

This package handles user registration, login, password hashing, and session tokens. It does not use JWTs. Tokens are random strings stored in a server-side ConcurrentHashMap.

PasswordUtil.java

Hashes and verifies passwords using SHA-256 with a random 16-byte salt. No external dependency — uses only java.security.MessageDigest and java.security.SecureRandom from the JDK.

Format stored in the database:

```
base64(salt) + ":" + base64(sha256(salt + password))
```

Method	What it does
hash(plaintext)	Generates a random salt, runs SHA-256, returns the combined string ready to store in the database.
verify(plaintext, stored)	Splits the stored string on ':', re-hashes the plaintext with the stored salt, compares using a constant-time loop to prevent timing attacks.

UserSession.java

Represents a logged-in user. Stored in SessionStore keyed by the token string. Fields: userId, role (ADMIN/CLIENT/CONSULTANT), name, email, createdAt timestamp. Sessions expire after 8 hours via isExpired().

SessionStore.java

In-memory token store. Uses ConcurrentHashMap for thread safety because the HTTP server runs on a thread pool of 10 threads. Sessions are intentionally ephemeral — when the server restarts, all users must log in again. This is appropriate for a course project and avoids the complexity of a sessions table with expiry cleanup.

Method	What it does
createSession(userId, role, name, email)	Generates a 32-byte cryptographically random URL-safe base64 token, stores the session, returns the token.
getSession(token)	Returns the session or null if missing or expired. Removes expired sessions lazily on access.
removeSession(token)	Called on logout.

AuthService.java

Business logic for registration and login. Bridges UserRepository (database) and SessionStore (memory).

Method	What it does
registerClient(name, email, password)	Validates: name not blank, email contains @, password at least 6 chars, email not already in database. Generates UUID, hashes password, saves user with role CLIENT.
registerConsultant(name, email, password, bio)	Same validation. Saves user with role CONSULTANT. Also saves consultant profile with status PENDING — all new consultants need admin approval.
createAdmin(name, email, password)	Checks if email already exists. If it does, silently returns (idempotent). Called once from AppContext on startup with ADMIN_EMAIL / ADMIN_PASSWORD env vars.
login(email, password)	Looks up user by email, throws the same error message whether not found or wrong password (never reveals whether the email exists). Calls PasswordUtil.verify(). Returns a session token.
logout(token)	Calls SessionStore.removeSession().

5. New Package: com.platform.http/

This package is the entire HTTP layer. It uses com.sun.net.httpserver.HttpServer which is built into the JDK — no additional web framework dependency is needed.

AppContext.java

The single wiring class for the entire Phase 2 backend. Created once in HttpServerLauncher.main() and shared with every handler.

Construction order (steps run on every server start):

- Step 1: Database.initialize() — connect to Postgres, run schema.sql, create tables
- Step 2: Instantiate all DB repositories
- Step 3: Instantiate auth components (SessionStore, AuthService)
- Step 4: Instantiate Phase 1 services pointing at the DB repositories
- Step 5: reloadFromDatabase() — loads all services and slots from DB back into ServiceCatalog and AvailabilityService
- Step 6: authService.createAdmin() — ensures the default admin account exists

Why reloadFromDatabase() is critical:
Phase 1 services (BookingService, AvailabilityService) depend on in-memory state. Without this reload step, all these in-memory maps would be empty on every restart even though data exists in Postgres. The reload bridges Phase 1 and Phase 2.

BaseHandler.java

Abstract base class for all 7 HTTP handlers. Provides shared utilities so handlers do not repeat boilerplate.

Utility group	What is provided
CORS	handleCors() adds Access-Control-Allow-Origin: * to every response and handles OPTIONS preflight automatically.
Response helpers	sendOk (200), sendCreated (201), send400, send401, send403, send404, send500. All serialize to JSON with Gson and set Content-Type.
Request helpers	readBody(), parseBody(), str(obj, field) — safely extracts a string field returning null if missing.
URL routing	subParts(ex, contextPath) strips the prefix and splits the path into segments. e.g. /bookings/BK-1/confirm gives [BK-1, confirm].
Auth	extractToken() reads Bearer token. requireAuth() returns session or sends 401. requireRole() also checks the role or sends 403.

HttpServerLauncher.java

The Phase 2 entry point. Main.java still works for the terminal demo. This launcher starts the HTTP server.

- Reads PORT env var (default 8080)
- Creates AppContext which wires everything
- Creates HttpServer bound to the port
- Registers all route contexts
- Sets a thread pool of 10 threads
- Calls server.start()

Run command: mvn compile exec:java -Dexec.mainClass=com.platform.http.HttpServerLauncher
Or in Docker: java -jar target/service-booking-platform-2.0.0.jar

dto/Dtos.java

All API response Data Transfer Objects in one file. Each DTO is a flat POJO with no circular references.

Why DTOs are necessary: the domain Service class holds a Consultant reference, and Consultant holds a List of Services. Gson would cause a StackOverflow on this circular graph. DTOs break the cycle by flattening the object into primitive fields only.

DTO class	Built from	Key fields
ServiceDto	Service	serviceId, title, description, durationMin, price, consultantId, consultantName
SlotDto	SlotRepository.SlotRow	id (UUID), consultantId, start, end, available

BookingDto	Booking	bookingId, clientId, clientName, serviceId, serviceTitle, consultantId, consultantName, slotStart, slotEnd, status, createdAt
ConsultantDto	Consultant	id, name, email, bio, registrationStatus
PaymentMethodDto	PaymentMethod	methodId, methodType, displayName (masked, e.g. CreditCard[****3456])
TransactionDto	PaymentTransaction or TransactionRow	id, bookingId, amount, status, timestamp

6. HTTP Handler Files

AuthHandler.java — /auth/*

Endpoint	Method	Auth needed	What it does
POST /auth/register/client	POST	None	Calls AuthService.registerClient(). Returns 201.
POST /auth/register/consultant	POST	None	Calls AuthService.registerConsultant(). Consultant starts PENDING.
POST /auth/login	POST	None	Returns token, userId, role, name, email on success.
POST /auth/logout	POST	Bearer token	Calls AuthService.logout().
GET /auth/me	GET	Bearer token	Returns current session's user info.

ServicesHandler.java — /services

Endpoint	Method	Auth needed	What it does
GET /services	GET	None (public)	Returns all services from ServiceCatalog.listAllServices() as ServiceDto array.
GET /services/{id}	GET	None (public)	Returns one service by id.
GET /services/{id}/slots	GET	None (public)	Returns available slots for the service's consultant from SlotRepository.
POST /services	POST	CONSULTANT	Creates a service, saves to ServiceRepository, adds to ServiceCatalog.

SlotsHandler.java — /slots (UC8)

Endpoint	Method	Auth needed	What it does

GET /slots	GET	CONSULTANT	Returns all slots for the logged-in consultant.
POST /slots	POST	CONSULTANT	Adds availability. ALWAYS auto-splits into 1-hour slots. A 6-hour block becomes 6 individual slots.
DELETE /slots/{uuid}	DELETE	CONSULTANT	Removes a slot. Blocked if is_available = false (reserved by a booking).

Critical design — auto-splitting:
POST /slots always calls AvailabilityService.addTimeSlotBlock(). This matches Phase 1 behaviour: a client booking one slot does not block the remaining hours of the consultant's day. A 1-hour range creates 1 slot; a 6-hour range creates 6 slots.

BookingsHandler.java — /bookings (UC2, UC3, UC4, UC9, UC10)

Endpoint	Auth	What it does
GET /bookings	Any role	CLIENT: own bookings. CONSULTANT: bookings for their services. ADMIN: all bookings.
GET /bookings/{id}	Any role	Single booking. Clients can only see their own.
POST /bookings	CLIENT	Creates booking. Validates slot available, calls BookingService.createBooking().
POST /bookings/{id}/confirm	CONSULTANT	UC9. Calls BookingService.confirmBooking(). Re-fetches from DB for response.
POST /bookings/{id}/reject	CONSULTANT	UC9. Calls BookingService.rejectBooking(). Releases the slot.
POST /bookings/{id}/complete	CONSULTANT	UC10. Calls BookingService.completeBooking(). Only works from PAID state.
POST /bookings/{id}/cancel	CLIENT or ADMIN	UC3. Calls BookingService.cancelBooking(). Triggers refund if booking was PAID.

State transition fix:
All action methods re-fetch the booking from DB after calling BookingService. BookingService mutates its own copy. The pre-call local booking reference still has the old state. Re-fetching ensures the HTTP response shows the correct new state (e.g. CONFIRMED not REQUESTED).

In-memory sync fix:
After POST /bookings creates a booking, the handler finds the matching TimeSlot inside AvailabilityService's in-memory map by start time and calls reserve() on it. Without this step, listAvailableSlots() would keep showing the slot as available in memory until the next restart.

PaymentsHandler.java — /payments (UC5, UC6, UC7)

Endpoint	Method	What it does
GET /payments/methods	GET	Loads saved payment methods from DB, syncs to in-memory PaymentMethodService.
POST /payments/methods	POST	Adds a method. Validates with Phase 1 validate() before saving.
PUT /payments/methods/{id}	PUT	UC6 update: replaces the method with the same id. Delete old + save new.
DELETE /payments/methods/{id}	DELETE	Removes from DB and in-memory service.
POST /payments/pay	POST	Processes payment for a CONFIRMED booking. Runs the Phase 1 CoR chain.
GET /payments/history	GET	Returns all transactions from PaymentTransactionRepository.

Payment flow: sets booking to PENDING_PAYMENT, runs ValidateMethodHandler → ProcessPaymentHandler (2-3 second delay, 10% random failure) → MarkPaidHandler (sets booking to PAID). Then updates booking status in DB and saves transaction to DB.

AdminHandler.java — /admin (UC11, UC12)

Endpoint	What it does
GET /admin/consultants	Lists all consultants from UserRepository.
POST /admin/consultants/{id}/approve	Sets registration_status to APPROVED in DB and on the in-memory Consultant object.
POST /admin/consultants/{id}/reject	Sets registration_status to REJECTED.
GET /admin/policies	Returns current policy class names using the new friendlyName() helper.
POST /admin/policies/cancellation	Sets cancellation policy: DEFAULT or STRICT (uses named StrictCancellationPolicy class).
POST /admin/policies/refund	Sets refund policy: DEFAULT (80%), FULL (100%), or NONE (0%, uses named NoRefundPolicy class).

Policy display fix: a friendlyName() helper method handles getSimpleName() gracefully. For named classes it returns the simple name. For any remaining anonymous classes or lambdas it returns the interface name + (custom) instead of the JVM-internal synthetic name.

ChatHandler.java — /chat (AI Customer Assistant)

Handles POST /chat { message: "..." } and returns { reply: "..." }. Requires a valid login token (any role).

Property	Value
Provider	hpc-ai.com
API endpoint	https://api.hpc-ai.com/inference/v1/chat/completions
Model	minimax/minimax-m2.5
Protocol	OpenAI-compatible chat completions format
Max tokens per response	300
HTTP client	Java built-in java.net.http.HttpClient (Java 11+, no extra Maven dependency)
API key	Read from OPENAI_API_KEY environment variable — never in source code

Privacy rules enforced in the system prompt:

- The AI is only given public platform information: booking steps, payment methods, cancellation policy, booking status definitions, and service titles and prices
- No user names, booking IDs, email addresses, or payment details are ever sent to the AI
- The AI has no connection to the database
- The AI is instructed not to claim access to user accounts or make up user-specific data
- All AI responses are HTML-escaped before rendering in the browser to prevent XSS

7. Database Schema

File: src/main/resources/schema.sql. All tables use CREATE TABLE IF NOT EXISTS so the file is safe to run on every server start.

Table	Primary key	Key columns
users	id (VARCHAR 36)	name, email (UNIQUE), password_hash, role (ADMIN/CLIENT/CONSULTANT)
consultants	user_id (FK → users.id)	bio, registration_status (PENDING/APPROVED/REJECTED)
services	id (VARCHAR 36)	title, description, duration_min, price, consultant_id (FK → users.id)
time_slots	id (UUID VARCHAR 36)	consultant_id (FK), start_time, end_time, is_available (BOOLEAN)
bookings	id (VARCHAR 36)	client_id (FK), service_id (FK), slot_id (FK), status, created_at
payment_methods	id (VARCHAR 36)	client_id (FK), method_type, details (JSON string)
payment_transactions	id (VARCHAR 36)	booking_id (FK), amount, status, timestamp

Data persistence: PostgreSQL data is stored in a named Docker volume called pgdata. Running docker-compose down keeps the data. Running docker-compose down -v deletes everything for a fresh start.

8. HTTP API Reference

All responses are application/json. Protected endpoints require Authorization: Bearer TOKEN header.

Auth endpoints — no token required

Endpoint	Request body
POST /auth/register/client	{ name, email, password }
POST /auth/register/consultant	{ name, email, password, bio }
POST /auth/login	{ email, password } → returns { token, userId, role, name, email }
POST /auth/logout	(token in Authorization header)
GET /auth/me	(token in Authorization header)

Client endpoints — CLIENT token required

Endpoint	Purpose
GET /services	Browse all services (public — no token needed)
GET /services/{id}/slots	Get available slots for a service (public)
GET /bookings	My booking history (UC4)
POST /bookings { serviceId, slotUuid }	Request a booking (UC2)
POST /bookings/{id}/cancel { reason }	Cancel a booking (UC3)
GET /payments/methods	View saved payment methods (UC6)
POST /payments/methods { type, fields }	Add a payment method (UC6)
PUT /payments/methods/{id} { type, fields }	Update a payment method (UC6)
DELETE /payments/methods/{id}	Remove a payment method (UC6)
POST /payments/pay { bookingId, methodId }	Process payment (UC5)
GET /payments/history	Payment history (UC7)
POST /chat { message }	Ask the AI assistant

Consultant endpoints — CONSULTANT token required

Endpoint	Purpose
GET /bookings	View all bookings for my services

POST /bookings/{id}/confirm	Accept a booking request (UC9)
POST /bookings/{id}/reject { reason }	Reject a booking request (UC9)
POST /bookings/{id}/complete	Mark session as completed (UC10)
GET /slots	View my availability (UC8)
POST /slots { start, end }	Add availability — auto-splits into 1-hour slots (UC8)
DELETE /slots/{uuid}	Remove an available slot (UC8)
POST /services { title, description, durationMin, price }	Add a new service

Admin endpoints — ADMIN token required

Endpoint	Purpose
GET /admin/consultants	List all consultant registrations (UC11)
POST /admin/consultants/{id}/approve	Approve a consultant (UC11)
POST /admin/consultants/{id}/reject	Reject a consultant (UC11)
GET /admin/policies	View current active policies (UC12)
POST /admin/policies/cancellation { type: DEFAULT|STRICT }	Set cancellation policy (UC12)
POST /admin/policies/refund { type: DEFAULT|FULL|NONE }	Set refund policy (UC12)
GET /bookings	View all bookings in the system

9. Frontend

The frontend is five HTML pages with shared CSS and shared JavaScript utilities. It runs inside an nginx container serving static files on port 3000. All backend communication goes through api.js.

File	Purpose
index.html	Redirect-only page. Sends logged-in users to their dashboard. Sends new visitors to login.html.
login.html	Login and registration for all three roles. Stores token in localStorage after login. Redirects by role.
client.html	Client dashboard: Browse Services (UC1), My Bookings (UC3/UC4), Payments (UC5/UC6/UC7), AI chatbot widget.
consultant.html	Consultant dashboard: Booking Requests (UC9/UC10), My Availability (UC8), My Services.

admin.html	Admin dashboard: Consultants (UC11), Policies (UC12), System Overview with stats and all bookings.
css/style.css	Shared stylesheet for all pages. Uses CSS custom properties for colours so the scheme can be changed in one place.
js/api.js	The only file that talks to the backend. Contains apiFetch() and a convenience api object with get, post, delete methods. Automatically adds the Bearer token. On 401 clears localStorage and redirects to login.
js/auth.js	Shared utilities: requireAuth(), logout(), switchTab(), showAlert(), badge(), fmtDate(). Loaded by all dashboard pages.
Dockerfile	nginx:alpine serving /usr/share/nginx/html. No build step needed.

Why JavaScript is required:
HTML forms can only submit as form-encoded data and cause a full page reload. They cannot send JSON, read JSON responses, or store a token for reuse across requests. JavaScript's fetch() is the only way to call a JSON API from a browser without a page reload.

AI Chatbot Widget (in client.html): A floating button in the bottom-right corner opens a chat panel. The user types a question and the frontend calls POST /chat. The backend calls hpc-ai.com and returns the reply. All AI text is HTML-escaped before rendering to prevent XSS.

10. Docker Setup

Backend/Dockerfile — two-stage build

- Stage 1 (maven:3.9.6-eclipse-temurin-17): copies pom.xml, runs mvn dependency:go-offline (cached layer so dependencies are not re-downloaded on every build), copies src/, runs mvn package -DskipTests. Produces a fat JAR at /build/target/service-booking-platform-2.0.0.jar.
- Stage 2 (eclipse-temurin:17-jre-alpine): copies only the JAR. ENTRYPOINT is java -jar app.jar. No Maven, no source code in the final image. Smaller and more secure.

Frontend/Dockerfile — single stage

FROM nginx:alpine. Copies all frontend files to /usr/share/nginx/html. Exposes port 80.

Root docker-compose.yml — three services

Service	Image	Port	Depends on
db	postgres:16-alpine	5432	nothing
backend	Built from ./Backend	8080	db (healthcheck — waits until Postgres is ready)

frontend	Built from ./Frontend	3000	backend

Health check: the db service has a healthcheck that runs pg_isready every 5 seconds. The backend uses depends_on: condition: service_healthy and will not start until Postgres is actually accepting connections. This prevents the backend from crashing and restarting repeatedly on a cold start.

Environment variables injected into the backend container:

- PORT=8080
- DB_URL=jdbc:postgresql://db:5432/booking_platform
- DB_USER=postgres / DB_PASSWORD=postgres
- ADMIN_EMAIL=admin@platform.com / ADMIN_PASSWORD=admin123
- OPENAI_API_KEY=${OPENAI_API_KEY} — read from the .env file in the project root

Backend/docker-compose.yml — development only

> **Warning:**
> The Backend/docker-compose.yml file is a development-only file for running just the database and backend without the frontend. It has a large warning comment at the top. Always use the ROOT docker-compose.yml for normal operation.

11. How to Run

With Docker (recommended)

```
git clone https://github.com/AliSheikhRabiei/EECS3311-Service-Booking-Platform.git
cd EECS3311-Service-Booking-Platform
cp .env.example .env    # then edit .env and add your OPENAI_API_KEY
docker-compose up --build
```

When you see '[Server] Ready', open http://localhost:3000 in your browser.

URL	What it is
http://localhost:3000/login.html	Frontend — use this
http://localhost:8080/health	API health check

Default admin account created automatically on first startup:

- Email: admin@platform.com
- Password: admin123

Without Docker (local development)

```
# Create the database first
psql -U postgres -c "CREATE DATABASE booking_platform;"
```

```
# Set environment variables (PowerShell)
$env:DB_URL = "jdbc:postgresql://localhost:5432/booking_platform"
$env:DB_USER = "postgres"
$env:DB_PASSWORD = "your_password"
$env:PORT = "8080"
$env:OPENAI_API_KEY = "sk-your-key"
```

```
# Run the Phase 2 HTTP server
cd Backend
mvn compile exec:java -Dexec.mainClass=com.platform.http.HttpServerLauncher
```

Run the original Phase 1 terminal demo (still works):

```
mvn exec:java
```

Run tests:

```
mvn test
```

12. Design Patterns — Unchanged from Phase 1

All four GoF patterns from Phase 1 are preserved in Phase 2. No pattern was removed, changed, or replaced. The HTTP layer calls through to the same pattern implementations.

Pattern	Files	How Phase 2 uses it
State	BookingState, AbstractBookingState, all 7 state classes	BookingsHandler calls BookingService which calls Booking which delegates to the current state object. The state enforces that a booking cannot be completed without being paid first.
Chain of Responsibility	ValidateMethodHandler, ProcessPaymentHandler, MarkPaidHandler	PaymentsHandler.processPayment() calls PaymentService.processPayment() which runs the handler chain. Each step can fail independently.
Strategy	CancellationPolicy, RefundPolicy, NotificationPolicy, PricingStrategy interfaces and all implementations	BookingService.cancelBooking() checks PolicyManager.getCancellationPolicy(). AdminHandler swaps policies at runtime via POST /admin/policies.
Singleton	PolicyManager.java	AppContext calls PolicyManager.getInstance(). All 7 handlers share the same policy state through this single instance.

13. Complete Summary of All Changes from Phase 1 to Phase 2

Every single file that was added, modified, or removed.

File	Change type	Summary
pom.xml	MODIFIED	Added Gson + PostgreSQL JDBC dependencies. Replaced maven-jar-plugin with maven-shade-plugin for fat JAR. Default mainClass changed to HttpServerLauncher.
payment/CreditCardMethod.java	MODIFIED	Added getCardNumber(), getExpiry(), getCvv() for DB serialization.
payment/DebitCardMethod.java	MODIFIED	Added getCardNumber(), getExpiry() for DB serialization.
payment/BankTransferMethod.java	MODIFIED	Added getAccountNumber(), getRoutingNumber() for DB serialization.
service/BookingService.java	MODIFIED	Booking ID changed from AtomicInteger counter to UUID to prevent primary key conflicts on restart.
policy/NoRefundPolicy.java	NEW	Named class for 0% refund. Replaces lambda — fixes admin policy display name.
policy/StrictCancellationPolicy.java	NEW	Named class for no-cancellations policy. Replaces anonymous class — fixes display name.
db/Database.java	NEW	JDBC connection factory and schema initialiser.
db/UserRepository.java	NEW	CRUD for users and consultants tables. Reconstructs domain objects.
db/ServiceRepository.java	NEW	CRUD for services table.
db/SlotRepository.java	NEW	CRUD for time_slots table.
db/DbBookingRepository.java	NEW	Extends BookingRepository. All operations go to PostgreSQL.
db/DbConsultantRepository.java	NEW	Extends ConsultantRepository. Delegates to UserRepository.
db/PaymentMethodRepository.java	NEW	CRUD for payment_methods table with JSON details column.
db/PaymentTransactionRepository.java	NEW	CRUD for payment_transactions table.
db/BookingStateFactory.java	NEW	Converts DB status strings to BookingState instances and back.
auth/PasswordUtil.java	NEW	SHA-256 + random salt password hashing with constant-time verification.
auth/UserSession.java	NEW	Logged-in user data holder with 8-hour expiry.

auth/SessionStore.java	NEW	ConcurrentHashMap-based in-memory token store.
auth/AuthService.java	NEW	Registration and login business logic.
http/AppContext.java	NEW	Single wiring class. Creates all repositories and services. Reloads DB data on startup.
http/BaseHandler.java	NEW	Abstract base for all handlers. CORS, auth, JSON helpers.
http/HttpServerLauncher.java	NEW	Phase 2 entry point. Starts HttpServer on port 8080. Registers all routes.
http/dto/Dtos.java	NEW	All API response DTOs. Flat POJOs that avoid circular reference Gson crashes.
http/handler/AuthHandler.java	NEW	Handles /auth/* endpoints for login, logout, registration.
http/handler/ServicesHandler.java	NEW	Handles /services — browse, get, get slots, add service.
http/handler/SlotsHandler.java	NEW	Handles /slots — list, add (auto-split), delete (UC8).
http/handler/BookingsHandler.java	NEW	Handles /bookings — full lifecycle UC2, UC3, UC4, UC9, UC10. Contains state fix and slot sync fix.
http/handler/PaymentsHandler.java	NEW	Handles /payments — UC5, UC6 (with update), UC7.
http/handler/AdminHandler.java	NEW	Handles /admin — UC11, UC12. Uses named policy classes for clean display names.
http/handler/ChatHandler.java	NEW	AI chatbot. Calls hpc-ai.com API. No personal data sent to AI.
src/main/resources/schema.sql	NEW	7-table PostgreSQL schema. Idempotent — safe to run on every startup.
Backend/Dockerfile	NEW	Two-stage Maven → JRE build. Produces fat JAR.
Backend/.env.example	NEW	Template documenting all required env vars including OPENAI_API_KEY.
Backend/docker-compose.yml	NEW (dev only)	Development-only compose file for db + backend. Has warning comment to use root file instead.
docker-compose.yml (root)	NEW	Three-container setup: db + backend + frontend. Single command deployment.
.env.example (root)	NEW	Template for the .env file containing the API key.
.gitignore (root)	NEW	Excludes .env, target/, .idea/, *.class, *.jar, *.log.
Frontend/index.html	NEW	Redirect page. Sends logged-in users to their dashboard.

Frontend/login.html	NEW	Login and registration for all three roles.
Frontend/client.html	NEW	Client dashboard with all 7 use cases + AI chatbot widget.
Frontend/consultant.html	NEW	Consultant dashboard with UC8, UC9, UC10 + service management.
Frontend/admin.html	NEW	Admin dashboard with UC11, UC12 + system overview.
Frontend/css/style.css	NEW	Shared stylesheet for all pages.
Frontend/js/api.js	NEW	All backend communication. One fetch wrapper with auto-auth and error handling.
Frontend/js/auth.js	NEW	Shared session management and utility functions.
Frontend/Dockerfile	NEW	nginx:alpine serving static files on port 80.
README.md	REPLACED	Phase 1 README replaced with full Phase 2 guide covering Docker, API, frontend, chatbot.
AI_CHATBOT_DOCUMENTATION.md	NEW	Required project deliverable. Documents chatbot integration, provider, model, privacy rules.
Phase2_ProjectGuide.md	NEW	This document. Complete reference guide for Phase 2.